

ATK-MD0700R 模块使用说明

高性能 7.0'RGBLCD 电容触摸屏模块

使用说明

正点原子

广州市星翼电子科技有限公司

修订历史

版本	日期	原因
V1.0	2023/06/05	第一次发布

目 录

1, 硬件连接.....	1
1.1 正点原子阿波罗 STM32F429 开发板	1
1.2 正点原子阿波罗 STM32F767 开发板	1
1.3 正点原子阿波罗 STM32H743 开发板.....	2
2, 实验功能.....	3
2.1 ATK-MD0700R-800480 模块测试实验.....	3
2.1.1 功能说明.....	3
2.1.2 源码解读.....	3
2.1.3 实验现象.....	11
2.2 ATK-MD0700R-1024600 模块测试实验.....	12
3, 其他.....	13

1，硬件连接

1.1 正点原子阿波罗 STM32F429 开发板

ATK-MD0700R 模块可直接与正点原子阿波罗 STM32F429 开发板板载的 TFTLCD 模块接口进行连接，具体的连接关系，如下表所示：

模块对应开发板	连接关系								
ATK-MD0700R 模块	VCC5	VCC5	R0	R1	R2	R3	R4	R5	R6
阿波罗 STM32F429 开发板	5V	5V	-	-	-	PH9	PH10	PH11	PH12
模块对应开发板	连接关系								
ATK-MD0700R 模块	R7	GND	G0	G1	G2	G3	G4	G5	G6
阿波罗 STM32F429 开发板	PG5	GND	-	-	PH13	PH14	PH15	PI0	PI1
模块对应开发板	连接关系								
ATK-MD0700R 模块	G7	GND	B0	B1	B2	B3	B4	B5	B6
阿波罗 STM32F429 开发板	PI2	GND	-	-	-	PG11	PI4	PI5	PI6
模块对应开发板	连接关系								
ATK-MD0700R 模块	B7	GND	CLK	HSYNC	VSNC	DE	BL	CS	MOSI
阿波罗 STM32F429 开发板	PI7	GND	PG7	PI10	PI9	PF10	PB5	PI8	PI3
模块对应开发板	连接关系								
ATK-MD0700R 模块	MISO	SCK	PEN	RESET	-	-	-	-	-
阿波罗 STM32F429 开发板	-	PH6	PH7	RESET	-	-	-	-	-

表 1.1.1 ATK-MD0700R 模块与阿波罗 STM32F429 开发板连接关系

1.2 正点原子阿波罗 STM32F767 开发板

ATK-MD0700R 模块可直接与正点原子阿波罗 STM32F767 开发板板载的 TFTLCD 模块接口进行连接，具体的连接关系，如下表所示：

模块对应开发板	连接关系								
ATK-MD0700R 模块	VCC5	VCC5	R0	R1	R2	R3	R4	R5	R6
阿波罗 STM32F767 开发板	5V	5V	-	-	-	PH9	PH10	PH11	PH12
模块对应开发板	连接关系								
ATK-MD0700R 模块	R7	GND	G0	G1	G2	G3	G4	G5	G6
阿波罗 STM32F767 开发板	PG5	GND	-	-	PH13	PH14	PH15	PI0	PI1
模块对应开发板	连接关系								
ATK-MD0700R 模块	G7	GND	B0	B1	B2	B3	B4	B5	B6
阿波罗 STM32F767 开发板	PI2	GND	-	-	-	PG11	PI4	PI5	PI6
模块对应开发板	连接关系								
ATK-MD0700R 模块	B7	GND	CLK	HSYNC	VSNC	DE	BL	CS	MOSI
阿波罗 STM32F767 开发板	PI7	GND	PG7	PI10	PI9	PF10	PB5	PI8	PI3

模块对应开发板	连接关系								
ATK-MD0700R 模块	MISO	SCK	PEN	RESET	-	-	-	-	-
阿波罗 STM32F767 开发板	-	PH6	PH7	RESET	-	-	-	-	-

表 1.2.1 ATK-MD0700R 模块与阿波罗 STM32F767 开发板连接关系

1.3 正点原子阿波罗 STM32H743 开发板

ATK-MD0700R 模块可直接与正点原子阿波罗 STM32H743 开发板板载的 TFTLCD 模块接口进行连接，具体的连接关系，如下表所示：

模块对应开发板	连接关系								
ATK-MD0700R 模块	VCC5	VCC5	R0	R1	R2	R3	R4	R5	R6
阿波罗 STM32H743 开发板	5V	5V	-	-	-	PH9	PH10	PH11	PH12
模块对应开发板	连接关系								
ATK-MD0700R 模块	R7	GND	G0	G1	G2	G3	G4	G5	G6
阿波罗 STM32H743 开发板	PG5	GND	-	-	PH13	PH14	PH15	PI0	PI1
模块对应开发板	连接关系								
ATK-MD0700R 模块	G7	GND	B0	B1	B2	B3	B4	B5	B6
阿波罗 STM32H743 开发板	PI2	GND	-	-	-	PG11	PI4	PI5	PI6
模块对应开发板	连接关系								
ATK-MD0700R 模块	B7	GND	CLK	HSYNC	VSYSNC	DE	BL	CS	MOSI
阿波罗 STM32H743 开发板	PI7	GND	PG7	PI10	PI9	PF10	PB5	PI8	PI3
模块对应开发板	连接关系								
ATK-MD0700R 模块	MISO	SCK	PEN	RESET	-	-	-	-	-
阿波罗 STM32H743 开发板	-	PH6	PH7	RESET	-	-	-	-	-

表 1.3.1 ATK-MD0700R 模块与阿波罗 STM32H743 开发板连接关系

2，实验功能

2.1 ATK-MD0700R-800480 模块测试实验

2.1.1 功能说明

在本实验中,开发板主控芯片通过 LTDC 接口与 ATK-MD0700R-800480 模块进行通讯,从而完成操作 ATK-MD0700R-800480 模块的 LCD 显示各种内容,同时通过模拟 IIC 接口与 ATK-MD0700R-800480 模块进行通讯,从而获取 ATK-MD0700R-800480 模块的触摸数据。

2.1.2 源码解读

打开本实验的工程文件夹,能够在./Drivers/BSP 目录下看到 ATK_RGBLCD 子文件夹,该文件夹中就包含了正点原子 RGBLCD 模块的驱动文件,如下图所示:

```
|-- atk_rgblcd.c
|-- atk_rgblcd.h
|-- atk_rgblcd_font.h
|-- atk_rgblcd_ltdc.c
|-- atk_rgblcd_ltdc.h
|-- atk_rgblcd_touch.h
|-- atk_rgblcd_touch_ftxx.c
|-- atk_rgblcd_touch_gtxx.c
|-- atk_rgblcd_touch_iic.c
|-- atk_rgblcd_touch_iic.h
```

图 2.1.2.1 正点原子 RGBLCD 模块驱动代码

2.1.2.1 正点原子 RGBLCD 模块接口驱动

在图 2.1.2.1 中,atk_rgblcd_ltdc.c 和 atk_rgblcd_ltdc.h 是开发板与正点原子 RGBLCD 模块通讯而使用的 LTDC 驱动文件,关于 LTDC 的驱动介绍,请查看正点原子各个开发板对应的开发指南中 LTDC 对应的章节。

2.1.2.2 正点原子 RGBLCD 模块字体文件

在图 2.1.2.1 中,atk_rgblcd_font.h 是驱动正点原子 RGBLCD 模块在 LCD 上显示 ASCII 字符时需要的字体取模文件,该文件支持字号为 12、16、24 和 32 的 ASCII 字符。

2.1.2.3 正点原子 RGBLCD 模块触摸接口驱动

在图 2.1.2.1 中,atk_rgblcd_touch_iic.c 和 atk_rgblcd_touch_iic.h 是开发板与正点原子 RGBLCD 模块通讯而使用的模拟 IIC 驱动文件,主要用于获取正点原子 RGBLCD 模块的触摸状态,关于模拟 IIC 的驱动介绍,请查看正点原子各个开发板对应的开发指南中模拟 IIC 对应的章节。

在图 2.1.2.1 中,atk_rgblcd_touch_ftxx.c 和 atk_rgblcd_touch_gtxx.c 是正点原子 RGBLCD 模块的触摸 IC 驱动文件,根据模块使用的不同触摸 IC 选择其中一个即可,关于触摸 IC 的驱动介绍,请查看正点原子各个开发板对应的开发指南中触摸屏对一个的章节。

2.1.2.4 正点原子 RGBLCD 模块驱动

在图 2.1.2.1 中,atk_rgblcd.c 和 atk_rgblcd.h 是正点原子 RGBLCD 模块的驱动文件,包

含了正点原子 RGBLCD 模块初始化、LCD 清屏、LCD 画点、LCD 画线、LCD 显示字符、LCD 显示字符串、LCD 显示数字等相关的正点原子 RGBLCD 模块操作 API 函数。函数比较多，下面仅介绍几个重要的 API 函数。

1. 函数 `atk_rgblcd_init()`

该函数用于初始化正点原子 RGBLCD 模块，具体的代码，如下所示：

```
/**
 * @brief   RGB LCD 模块初始化
 * @param   无
 * @retval  ATK_RGBLCD_EOK: RGB LCD 模块初始化成功
 *          ATK_RGBLCD_ERROR: RGB LCD 模块初始化失败
 */
uint8_t atk_rgblcd_init(void)
{
    uint8_t id;
    uint8_t ret;

    /* RGB LCD 模块 ID 获取 */
    id = atk_rgblcd_get_id();

    /* RGB LCD 模块参数初始化 */
    ret = atk_rgblcd_setup_param_by_id(id);
    if (ret != ATK_RGBLCD_EOK)
    {
        return ATK_RGBLCD_ERROR;
    }

    /* RGB LCD 模块硬件初始化 */
    atk_rgblcd_hw_init();

    /* RGB LCD 模块 LTDC 接口初始化 */
    atk_rgblcd_ltdc_init(ATK_RGBLCD_LCD_RAW_WIDTH,
                        ATK_RGBLCD_LCD_RAW_HEIGHT,
                        &g_atk_rgblcd_sta.param->timing);
    g_atk_rgblcd_sta.fb = (uint16_t *)ATK_RGBLCD_LTDC_LAYER_FB_ADDR;

#ifdef ATK_RGBLCD_USING_DMA2D != 0
    /* 初始化 DMA2D */
    atk_rgblcd_dma2d_init();
#endif

    atk_rgblcd_set_disp_dir(ATK_RGBLCD_LCD_DISP_DIR_0);
    atk_rgblcd_clear(ATK_RGBLCD_WHITE);
    atk_rgblcd_backlight_on();
}
```

```
#if (ATK_RGBLCD_USING_TOUCH != 0)

    /* 初始化触摸 */
    ret = atk_rgblcd_touch_init(g_atk_rgblcd_sta.param->touch_type);
    if (ret != ATK_RGBLCD_TOUCH_EOK)
    {
        return ATK_RGBLCD_ERROR;
    }
#endif

return ATK_RGBLCD_EOK;
}
```

从上面的代码中可以看出，函数 `atk_rgblcd_init()` 初始化正点原子 RGBLCD 模块主要就是先获取正点原子 RGBLCD 模块的 ID，不同的 ID 对应不同的正点原子 RGBLCD 模块，然后根据正点原子 RGBLCD 模块的 ID 来配置初始化 LTDC。通过还通过宏定义 `ATK_RGBLCD_USING_DMA2D` 来使能或禁用 DMA2D 的相关驱动，若使能了 DMA2D 的相关驱动，则还会调用 `atk_rgblcd_dma2d_init()` 来初始化 DMA2D，同时也通过宏定义 `ATK_RGBLCD_USING_TOUCH` 来使能或禁用正点原子 RGBLCD 模块的触摸驱动，若使能了正点原子 RGBLCD 模块的触摸驱动，还会调用函数 `atk_rgblcd_touch_init()` 进行触摸的相关初始化，这个函数在下面会进行介绍。

2. 函数 `atk_rgblcd_draw_point()`

该函数用于在正点原子 RGBLCD 模块的 LCD 上画一个点，理论上只要通过该函数就能够完成对正点原子 RGBLCD 模块 LCD 的所有显示操作，该函数的具体代码，如下所示：

```
/**
 * @brief   RGB LCD 模块 LCD 画点
 * @param   x: 待画点的 x 坐标
 *          y: 待画点的 y 坐标
 *          color: 待画点的颜色
 * @retval  无
 */
void atk_rgblcd_draw_point(uint16_t x, uint16_t y, uint16_t color)
{
    atk_rgblcd_pos_transform(&x, &y);
    ATK_RGBLCD_FB[y * ATK_RGBLCD_LCD_RAW_WIDTH + x] = color;
}
```

从上面的代码中可以看出，在正点原子 RGBLCD 模块的 LCD 上画点就是将待画点的颜色数据写入显存的指定位置即可。

3. 函数 `atk_rgblcd_fill()`

该函数用于对正点原子 RGBLCD 模块 LCD 的某一区域填充指定的单一颜色，该函数的具体代码，如下所示：

```
/**
 * @brief   RGB LCD 模块 LCD 区域填充
 * @param   xs: 区域起始 x 坐标
 *          ys: 区域起始 y 坐标
 *          xe: 区域终止 x 坐标
```

```

*           ye: 区域终止 y 坐标
*           color: 区域填充颜色
* @retval 无
*/
void atk_rgblcd_fill(  uint16_t xs,
                      uint16_t ys,
                      uint16_t xe,
                      uint16_t ye,
                      uint16_t color)
{
    if (xe >= ATK_RGBLCD_LCD_WIDTH)
    {
        xe = ATK_RGBLCD_LCD_WIDTH - 1;
    }
    if (ye >= ATK_RGBLCD_LCD_HEIGHT)
    {
        ye = ATK_RGBLCD_LCD_HEIGHT - 1;
    }

    #if (ATK_RGBLCD_USING_DMA2D != 0)
        atk_rgblcd_dma2d_fill(xs, ys, xe, ye, color);
    #else
        uint16_t x_index;
        uint16_t y_index;

        for (y_index=ys; y_index<ye + 1; y_index++)
        {
            for (x_index=xs; x_index<xe + 1; x_index++)
            {
                atk_rgblcd_pos_transform(&x_index, &y_index);
                ATK_RGBLCD_FB[y_index * ATK_RGBLCD_LCD_RAW_WIDTH + x_index] = color;
            }
        }
    #endif
}

```

从上面的代码中可以函数，该函数也是将待画颜色数据写入显存的对应位置，但该函数也可使用 DMA2D 来进行区域填充，这能大幅提高运行的效率。

2.1.2.5 正点原子 RGBLCD 模块触摸驱动

在图 2.1.2.1 中, `atk_rgblcd_touch_ftxx.c` 和 `atk_rgblcd_touch_gtxx.c` 是正点原子 RGBLCD 模块的触摸驱动文件，包含了正点原子 RGBLCD 模块触摸初始化和扫描等相关的正点原子 RGBLCD 模块触摸 API 函数，分别适用于两类不同的触摸 IC，这里与 `atk_rgblcd_touch_gtxx.c` 文件为例进行介绍，`atk_rgblcd_touch_ftxx.c` 也是类似的。函数比较多，下面仅介绍几个重要的 API 函数。

1. 函数 `atk_rgblcd_touch_init()`

该函数用于初始化正点原子 RGBLCD 模块的触摸，具体的代码，如下所示：

```
/**
 * @brief   fRGB LCD 模块触摸初始化
 * @param   无
 * @retval  ATK_RGBLCD_TOUCH_EOK: RGB LCD 模块触摸初始化成功
 *          ATK_RGBLCD_TOUCH_ERROR: RGB LCD 模块触摸初始化失败
 */
uint8_t atk_rgblcd_touch_init(atk_rgblcd_touch_type_t type)
{
    char pid[5];

    if (type != ATK_RGBLCD_TOUCH_TYPE_GTXX)
    {
        return ATK_RGBLCD_TOUCH_ERROR;
    }

    atk_rgblcd_touch_hw_init();
    atk_rgblcd_touch_hw_reset(ATK_RGBLCD_TOUCH_IIC_ADDR_14);
    atk_rgblcd_touch_iic_init();
    atk_rgblcd_touch_get_pid(pid);
    if ( (strcmp(pid, ATK_RGBLCD_TOUCH_PID) != 0) &&
        (strcmp(pid, ATK_RGBLCD_TOUCH_PID1) != 0) &&
        (strcmp(pid, ATK_RGBLCD_TOUCH_PID2) != 0) &&
        (strcmp(pid, ATK_RGBLCD_TOUCH_PID3) != 0))
    {
        return ATK_RGBLCD_TOUCH_ERROR;
    }
    atk_rgblcd_touch_sw_reset();

    return ATK_RGBLCD_TOUCH_EOK;
}
```

从上面的代码中可以看出，函数 `atk_rgblcd_touch_init()` 初始化正点原子 RGBLCD 模块的触摸功能主要就是初始化与正点原子 RGBLCD 模块触摸相关的模拟 IIC 通讯接口，模拟 IIC 通讯接口初始化完成后就可以尝试通过模拟 IIC 读取正点原子 RGBLCD 模块触摸的 PID，以此判断与正点原子 RGBLCD 模块触摸的通讯是否正常初始化。

2. 函数 `atk_rgblcd_touch_scan()`

该函数用于扫描正点原子 RGBLCD 模块的触摸，具体的代码，如下所示：

```
/**
 * @brief   RGB LCD 模块触摸扫描
 * @note    连续调用间隔需大于 10ms
 * @param   point: 扫描到的触摸点信息
 *          cnt: 需要扫描的触摸点数量 (1~ATK_RGBLCD_TOUCH_TP_MAX)
 * @retval  0: 没有扫描到触摸点
 *          其他: 实际获取到的触摸点信息数量
 */
```

```
*/
uint8_t atk_rgblcd_touch_scan(atk_rgblcd_touch_point_t *point, uint8_t cnt)
{
    uint8_t tp_info;
    uint8_t tp_cnt;
    uint8_t point_index;
    atk_rgblcd_lcd_disp_dir_t dir;
    uint8_t tpn_info[6];
    atk_rgblcd_touch_point_t point_raw;

    if ((cnt == 0) || (cnt > ATK_RGBLCD_TOUCH_TP_MAX))
    {
        return 0;
    }

    for (point_index=0; point_index<cnt; point_index++)
    {
        if (&point[point_index] == NULL)
        {
            return 0;
        }
    }

    atk_rgblcd_touch_read_reg( ATK_RGBLCD_TOUCH_REG_TPINFO,
                              &tp_info,
                              sizeof(tp_info));
    if ((tp_info & ATK_RGBLCD_TOUCH_TPINFO_MASK_STA) ==
        ATK_RGBLCD_TOUCH_TPINFO_MASK_STA)
    {
        tp_cnt = tp_info & ATK_RGBLCD_TOUCH_TPINFO_MASK_CNT;
        tp_cnt = (cnt < tp_cnt) ? cnt : tp_cnt;

        for (point_index=0; point_index<tp_cnt; point_index++)
        {
            atk_rgblcd_touch_read_reg(g_atk_rgblcd_touch_tp_reg[point_index],
                                      tpn_info,
                                      sizeof(tpn_info));

            point_raw.x = (uint16_t)(tpn_info[1] << 8) | tpn_info[0];
            point_raw.y = (uint16_t)(tpn_info[3] << 8) | tpn_info[2];
            point_raw.size = (uint16_t)(tpn_info[5] << 8) | tpn_info[4];

            dir = atk_rgblcd_get_disp_dir();
            switch (dir)
            {
```

```
case ATK_RGBLCD_LCD_DISP_DIR_0:
{
    point[point_index].x = point_raw.x;
    point[point_index].y = point_raw.y;
    break;
}
case ATK_RGBLCD_LCD_DISP_DIR_90:
{
    point[point_index].x = point_raw.y;
    point[point_index].y = atk_rgblcd_get_lcd_height() -
        point_raw.x;
    break;
}
case ATK_RGBLCD_LCD_DISP_DIR_180:
{
    point[point_index].x = atk_rgblcd_get_lcd_width() -
        point_raw.x;
    point[point_index].y = atk_rgblcd_get_lcd_height() -
        point_raw.y;
    break;
}
case ATK_RGBLCD_LCD_DISP_DIR_270:
{
    point[point_index].x = atk_rgblcd_get_lcd_width() -
        point_raw.y;
    point[point_index].y = point_raw.x;
    break;
}
}

point[point_index].size = point_raw.size;
}

tp_info = 0;
atk_rgblcd_touch_write_reg(ATK_RGBLCD_TOUCH_REG_TPINFO,
    &tp_info,
    sizeof(tp_info));

return tp_cnt;
}
else
{
    return 0;
}
```

```
}
```

从上面的代码中可以看出，函数 `atk_rgblcd_touch_scan()` 首先会判断是否有触摸点被按下，如果有再获取有多少个触摸点被按下，然后分别获取被按下的触摸点的信息（X 坐标、Y 坐标、大小），最后根据屏幕的旋转方向，计算出并返回实际的触摸点坐标信息。

2.1.2.6 实验测试代码

实验的测试代码为文件 `demo.c`，在工程目录下的 `User` 子目录中。测试代码的入口函数为 `demo_run()`，具体的代码，如下所示：

```
/**
 * @brief  例程演示入口函数
 * @param  无
 * @retval 无
 */
void demo_run(void)
{
    uint8_t ret;
    uint16_t pid;

    /* 初始化 RGB LCD 模块 */
    ret = atk_rgblcd_init();
    /* 获取 RGB LCD 模块 PID */
    pid = atk_rgblcd_get_pid();
    if ((ret != 0) || (pid != ATK_RGBLCD_PID_7084))
    {
        printf("ATK-MD0700R-800480 connection error!\r\n");
        while (1)
        {
            LED0_TOGGLE();
            delay_ms(200);
        }
    }

    /* RGB LCD 模块 LCD 清屏 */
    atk_rgblcd_clear(ATK_RGBLCD_WHITE);
    /* RGB LCD 模块 LCD 显示字符串 */
    atk_rgblcd_show_string(10,
                           10, atk_rgblcd_get_lcd_width(),
                           32,
                           "STM32",
                           ATK_RGBLCD_LCD_FONT_32,
                           ATK_RGBLCD_RED);
    atk_rgblcd_show_string(10,
                           42,
                           atk_rgblcd_get_lcd_width(),
                           24,
```

```

        "ATK-MD0700R-800480",
        ATK_RGBLCD_LCD_FONT_24,
        ATK_RGBLCD_RED);

atk_rgblcd_show_string( 10,
                        66,
                        atk_rgblcd_get_lcd_width(),
                        16,
                        "ATOM@ALIENTEK",
                        ATK_RGBLCD_LCD_FONT_16,
                        ATK_RGBLCD_RED);

while (1)
{
    /* 演示立方体 3D 旋转 */
    demo_show_cube();
}
}

```

从上面的代码中可以看出，整个测试代码的逻辑还是比较简单的，就是在 ATK-MD0700R-800480 模块的 LCD 上显示了一些实验信息，然后就调用函数 demo_show_cube() 显示立方体 3D 旋转的演示，函数 demo_show_cube() 实际上就是通过 LCD 画线函数在 ATK-MD0700R-800480 模块的 LCD 显示屏上画线段，画线的本质实际上也就是画点，同时根据扫描的触摸坐标值，实时的修改立方体的位置。

2.1.3 实验现象

将 ATK-MD0700R-800480 模块按照第一节“硬件连接”中介绍的连接方式与开发板连接，并将实验代码编译烧录至开发板中，在 ATK-MD0700R-800480 模块初始化前，会先通过串口显示本实验的相关信息，如下图所示：

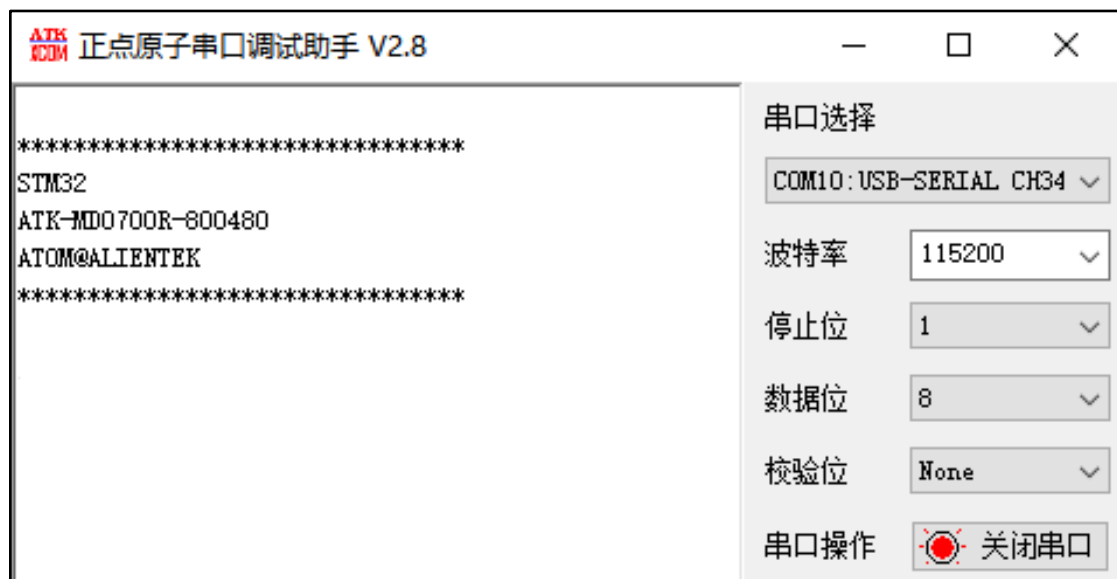


图 2.1.3.1 串口调试助手显示内容

初始化通过后，会在 ATK-MD0700R-800480 模块的 LCD 上显示一些实验信息，和立方体 3D 旋转的演示，如下图所示：

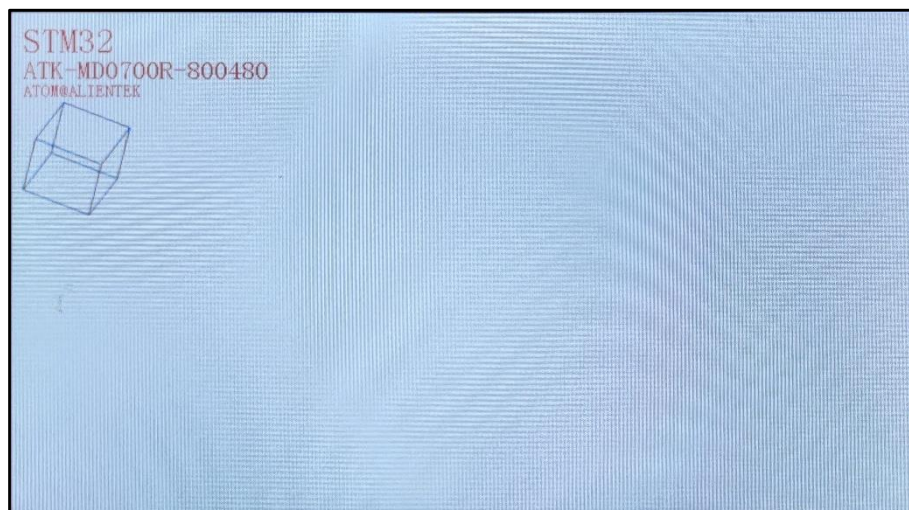


图 2.1.3.2 ATK-MD0700R-800480 模块 LCD 显示立方体 3D 旋转演示等信息
此时通过触摸屏幕，可以实时修改立方体的位置，如下图所示：

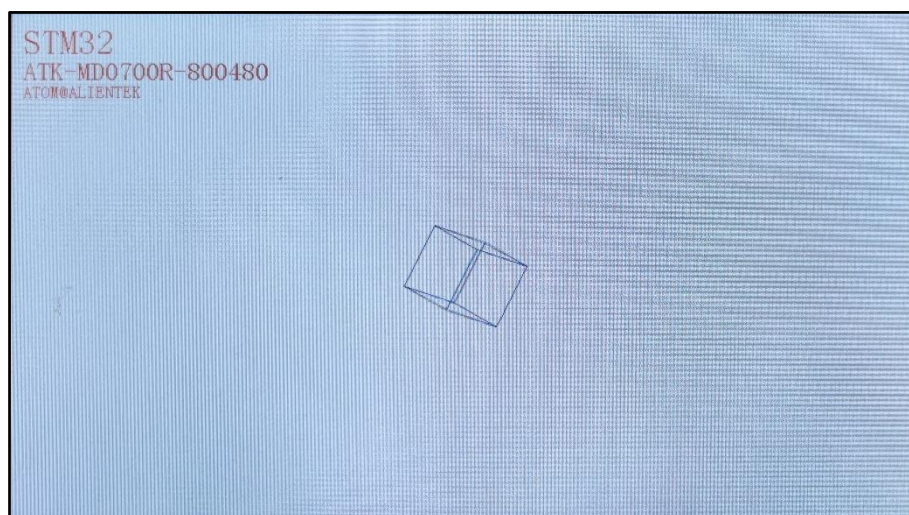


图 2.1.3.3 触摸修改立方体位置

2.2 ATK-MD0700R-1024600 模块测试实验

本实验的 ATK-MD0700R-1024600 模块的测试例程，大致内容与 ATK-MD0700R-1024600 模块测试实验类似，请查看第 2.1 小节“ATK-MD0700R-1024600 模块测试实验”。

3，其他

1、购买地址：

天猫：<https://zhengdianyuanzi.tmall.com>

淘宝：<https://openedv.taobao.com>

2、资料下载

ATK-MD0700R-800480 模块资料下载地址：<http://www.openedv.com/docs/modules/lcd/7-RGB%20LCD-%20800480.html>

ATK-MD0700R-1024600 模块资料下载地址：<http://www.openedv.com/docs/modules/lcd/7-RGBLCD-1024600.html>

3、技术支持

公司网址：www.alientek.com

技术论坛：<http://www.openedv.com/forum.php>

在线教学：www.yuanzige.com

B 站视频：<https://space.bilibili.com/394620890>

传真：020-36773971

电话：020-38271790

